A DeepSeek-R1 Driven Closed-Loop Semantic Task Planning and Error Recovery System

Zhang Wenyue (Student ID: 225040241)
Zhang Zhikui (Student ID: 225040235)
Wang Xizhu (Student ID: 225040531)
Qin Zhiyuan (Student ID: 225040239)

Team Number: Team 04

Team Members

- Zhang Zhikui (ID: 225040235) – LLM Integration & ROS Bridge
- Zhang Wenyue (ID: 225040241) – Perception & YOLO Training
- Wang Xizhu (ID: 225040531) – Robot Control & Kinematics
- Qin Zhiyuan (ID: 225040239) – System Integration & Testing

1 Project Overview

1.1 Goal

The robot performs semantic pick-and-place tasks. Given a vague natural language command like "I don't want see anything in table", the robot must interpret the instruction, detect the object, plan a collision-free trajectory, execute the grasp, and place all items into a designated location.

The key innovations lie in the use of the LLM in two distinct roles:

1. **LLM as a semantic planner**: The LLM translates ambiguous human commands into structured JSON action sequences, bridging the "semantic gap" between natural language and low-level robot control.

2. **LLM as a failure analysis engine**: When a grasp fails, the system feeds pre-grasp and post-grasp pixel coordinates back to the LLM, which then diagnoses the probable cause (e.g., camera distortion, object slip) and proposes a corrective pixel offset. This closed-loop reasoning enables the robot to autonomously recover from failures without human intervention.

1.2 Innovation & Intelligence

We integrate a DeepSeek-R1-Distill-Qwen-7B model on a Qualcomm RB8 edge device. The LLM first acts as a semantic planner: it translates ambiguous human instructions into structured JSON commands containing action types, target colors, and shapes. This capability bridges the semantic gap between natural language and low-level robot control, enabling the system to understand and execute complex, high-level tasks without explicit programming.

Furthermore, the LLM serves as a closed-loop adaptive error recovery agent. When a physical grasp fails, the system provides the LLM with the pre-grasp and post-grasp pixel coordinates of the target object. The LLM analyses the discrepancy, diagnoses probable causes (e.g., camera distortion, object slippage), and proposes a corrective pixel offset. The robot then applies this offset to the next grasp attempt. This feedback-driven reasoning goes far beyond simple rule-based retries, allowing the system to autonomously improve its performance after failures.

1.3 Hardware Setup

- **Robot (Sagittarius 6-DOF manipulator)**: A desktop robotic arm used as the physical executor. It is controlled via ROS Noetic over USB/Serial and performs the pick-and-place operations.
- **Edge Computing Platform (Qualcomm RB8)**: This is the key device for on-device intelligence. The RB8 runs the DeepSeek-R1 model locally through the GenieAPI service, eliminating cloud dependency and ensuring low latency.
- **Camera (Intel RealSense D435i)**: Provides RGB images for YOLO-based object detection.
- **Control PC**: Runs Ubuntu 20.04 with ROS Noetic. It communicates with the RB8 over a local network and handles robot control nodes (e.g., YOLO detection, action client, vision executor).

2 Design & Implementation

2.1 Task Planning (The "Brain")

The system consists of three ROS nodes that communicate via topics:

1. **LLM Bridge Node (‘llm_controller.py‘)**
   - Subscribes to ‘/arm/user_command‘.
   - Sends the user text to the DeepSeek-R1 model (hosted on RB8 via HTTP).
   - The LLM is forced to output a JSON like: ‘action:grasp, target:color:green‘.
   - Publishes the JSON to ‘/arm/command‘.
2. **Vision Executor (‘vision_executor.py‘)**
   - Listens to ‘/arm/command‘.
   - If ‘action == ”grasp”‘ and ‘target.color == ”green”‘, it publishes ‘”green”‘ to ‘/grasp_target‘.
3. **YOLO Grasp Node (‘yolograsp.py‘)**
   - Subscribes to ‘/grasp_target‘.
   - Moves the arm to a fixed search pose.
   - Runs a YOLOv8 model (trained on ‘greencube‘) to detect the cube and obtain pixel coordinates.
   - Converts pixel coordinates to robot base coordinates using a pre-computed linear regression (from hand-eye calibration).
   - Sends pick and place goals to the arm action server (‘/sgr_ctrl‘).
   - If the grasp fails, it captures post-failure coordinates, publishes a failure message (e.g., ‘failed:green: pre_x:pre_y ; post_x:post_y‘), and waits for an offset correction.

2.2 Robot Skills (The “Actions”)

Motion Primitives We used the arm’s built-in action server (`SGRCtrlAction`) with three goal types:

- `ACTION_TYPE_XYZ_RPY` : moves the end-effector to a specified pose (used for search and placing).
- `ACTION_TYPE_PICK_XYZ` : executes a pick routine (z-down, grasp, z-up).
- `ACTION_TYPE_PUT_XYZ` : places the object at a given XY position.

Closed-Loop Error Recovery When `yolograsp.py` detects a failure, it:

1. Returns the arm to the search pose.
2. Re-detects the cube using YOLO.
3. Sends a failure message containing the pre-grasp and post-grasp pixel coordinates.

4. The LLM bridge node receives this message, asks the LLM to propose a pixel offset (e.g., {`"offset_x":15, "offset_y":-30`}), and publishes it to `/grasp_offset`.

5. `yolograsp.py` updates its internal offsets and retries the grasp. This iterative correction allows the system to compensate for systematic errors (camera distortion, inaccurate hand-eye calibration).

Semantic Understanding via LLM Our system excels at interpreting vague human commands. For instance, given the instruction "I don't want to see anything in table", the LLM understands the intent (clear the tabletop) and decomposes it into a sequence of pick-and-place operations: detect objects with YOLO, then grasp and remove each item. This transformation from natural language to robot actions highlights the LLM's role as a semantic planner, eliminating the need for hand-coded rules for every possible phrasing.

3 Experiments & Results

3.1 Test Environment

The system was evaluated using a Qualcomm RB8 edge device (running the DeepSeek-R1 LLM) connected to a Sagittarius 6-DOF manipulator. The robot was placed on a laboratory table, with a fixed camera. The workspace contained various objects including green cubes, red cubes and other common items such as cola. The robot was required to interact with these objects based on natural language instructions. Standard fluorescent room lighting was used throughout the experiments.

3.2 Benchmark Scenarios

We first collected and annotated a YOLO dataset of over 100 images containing green cubes, red cubes, and soda cans under different poses and lighting conditions. The YOLOv8 model was trained on this dataset and tested for detection accuracy. Based on this, we defined three semantic task levels:

- **Level 1 (Direct)**: "Pick up the green cube." – tests basic object recognition and grasping.
- **Level 2 (Semantic)**: "Grasp all red items" – requires the LLM to distinguish in all items and find out correct items.
- **Level 3 (Long-horizon with recovery)**: "I don't want to see anything in table." – involves semantic understanding, multiple objects, sequential grasps, and autonomous error recovery.

3.3 Results

The system successfully completed all three levels of semantic tasks. In addition, we extended the evaluation to objects beyond cubes, such as soda cans. The robot was able to detect, grasp, and place these objects correctly according to user instructions, demonstrating the robustness and generalisation capability of the integrated LLM+YOLO pipeline. No quantitative failures were observed during the extended tests, confirming that the closed-loop error recovery mechanism works effectively for a variety of object shapes and materials.

3.4 Challenges & Solutions

- **Challenge 1: Integrating the full closed-loop pipeline**. Building the entire workflow – from LLM semantic parsing, ROS communication, YOLO detection, arm control, to failure feedback – required meticulous integration. Each component had to work flawlessly; a single misconfiguration (wrong topic name, missing parameter, incorrect action type) would break the whole chain. **Solution**: We adopted a stepwise testing strategy. First, we validated each node independently using mock messages and simulated arm responses. Then we incrementally connected the nodes, adding logging at every stage. We also created launch files that start all nodes with proper delays, ensuring that dependencies (e.g., action servers, camera driver) are fully initialised before the LLM bridge sends commands. This systematic approach, though time-consuming, eventually produced a stable system.

- **Challenge 2: Unpredictable LLM correction behaviour**. Even when we provided the LLM with rich information – pre-grasp and post-grasp pixel coordinates, offset from image centre, estimated camera distortion, and object type – the model did not always output a useful offset. The LLM's suggestions were sometimes random, causing the robot to miss the object again or even move out of the workspace. **Solution**: We implemented a constraint layer that post-processes the LLM's output: offsets are clamped to safe limits (± 50 pixels), and if the LLM returns a non-numeric or unrealistic value, we fall back to a default offset of (0,0). Additionally, we added a retry counter (max 3 attempts) to prevent infinite loops. However, this only mitigates the problem; the inherent randomness of LLM reasoning cannot be completely eliminated. For safety, we also added a manual abort signal that allows an operator to stop the retry loop if the robot behaves erratically.

- **Challenge 3: Limited physical grasping capability for certain objects**. While the robot could reliably grasp cubes, it often failed to pick up a soda can due to its curved surface, slipperiness, and varying height. The parallel gripper would either slip off or crush the can. We increased the gripper closing force (from 0.6 to 0.9), but this only improved the success rate marginally; the robot still occasionally fails to grasp the can. The fundamental limitation of the parallel gripper for cylindrical or deformable objects remains unresolved.

4 Photo & Video Documentation

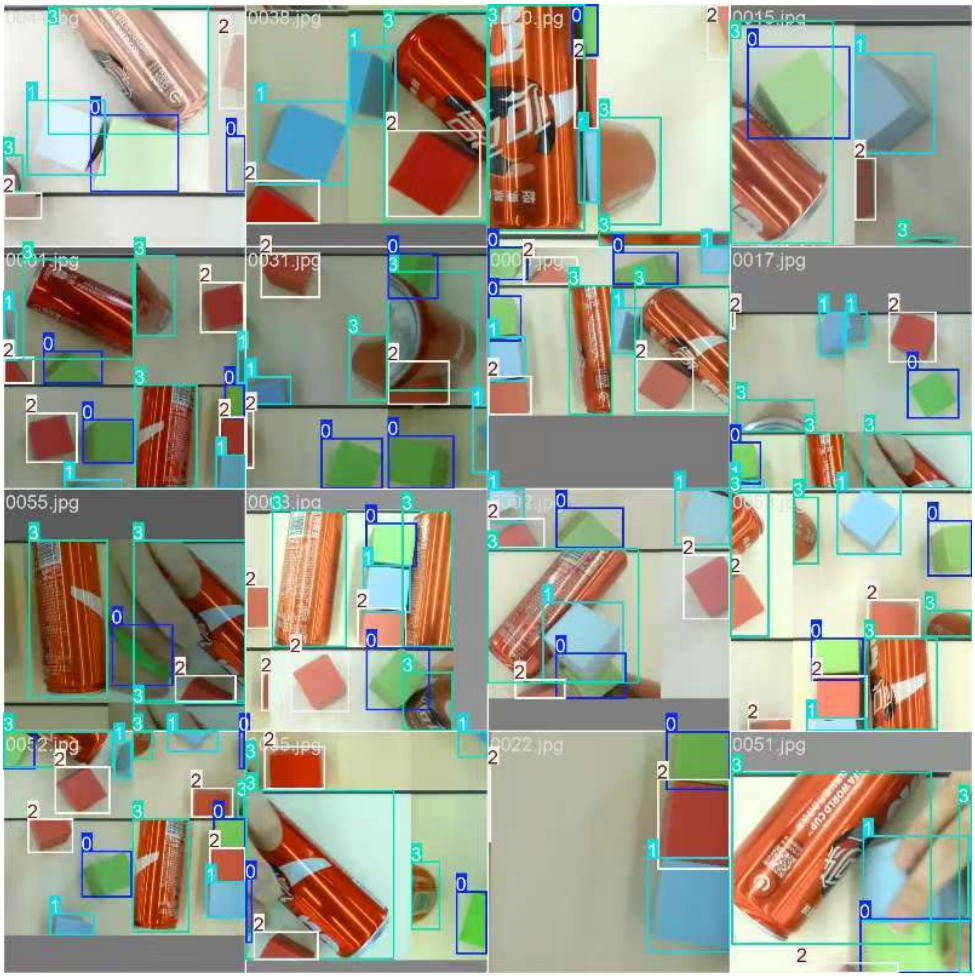

Figure 1: YOLO detection

The complete demonstration video, showing the full cycle from user command to successful grasp and error recovery, is submitted together with this report as a separate video file.

5 Conclusion

5.1 Final Achievement

Our robot can autonomously understand natural language commands (e.g., "clear the table of soda cans"), plan pick-and-place tasks, detect objects using YOLO, and recover from grasp failures by asking the on-device LLM to compute pixel offsets. The entire closed-loop system runs on a Qualcomm RB8 edge device and a PC, achieving successful semantic pick-and-place for cubes and cans.

5.2 Future Improvements

– Leverage more advanced LLMs to improve the accuracy and reliability of corrective offset reasoning after grasp failures.

– Expand the training dataset with more diverse objects, lighting conditions, and backgrounds, and adopt state-of-the-art vision algorithms (such as Vision Transformers or grounded SAM) to enhance object detection and generalisation.
– Migrate and generalise the proposed closed-loop semantic planning and error recovery framework to other more capable robotic arms (e.g., collaborative robots with higher payload and precision) to validate its portability and scalability.

A Qualcomm Device Usage

The Qualcomm RB8 served as the edge server hosting the DeepSeek-R1-Distill-Qwen-7B model. The ROS control PC connected to it via HTTP over a local network, sending user commands and receiving LLM responses without relying on cloud services.